

Raj Mankar

Professor Rubin

AP Calculus BC

Final Project: Just Because; Neural Network Music Genre Classifier

What I Chose and Why

For my “Just Because” project for Calc BC, I chose to learn how to build and train a neural network from scratch using *Python*, *TensorFlow*, and *Keras*. Specifically, I built a *music genre classification model* that takes a 30-second audio clip, extracts 58 numerical features from it, and predicts which of 10 genres it belongs to: blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, or rock.

I chose this project for a few personal reasons. Firstly, I plan to study Data Science in college, and neural networks are one of the most important tools in the field, so I decided to start it now for good exposure before campus. Also, there's a lot of hidden calculus sitting inside of a neural network, so I could pull out some really interesting stuff; along the theme, music classification is a very noble first project for a neural network with Tensorflow, the “*Hello World*” of the neural network line of work is using the *MNIST* database to classify handdrawn numbers to which number they are, which is boring and requires extensive coding/structuring — personally, an odd final for Calc BC.

How I Learned It

The learning process was not linear. It started around AP exam season, when I had very little time I watched a few YouTube videos explaining what a neural network conceptually was, and watched some demos of people building them. That gave me enough of a mental model to know what I was trying to build, but, the bulk would be after exams.

After exams, and with pressure easing, I started working in Google Colab in earnest. I learned how to use TensorFlow and Keras through a combo of YouTube, reading docs, and a stupid amount of trial and error. In the end, I had to get some help from my friend who's pursuing computer science in order to work through some specific problems I had. The process of actually building the project; writing a cell, running it, getting an error, figuring out what to do from there, fixing it, and repeating, taught me more than any YouTube video could.

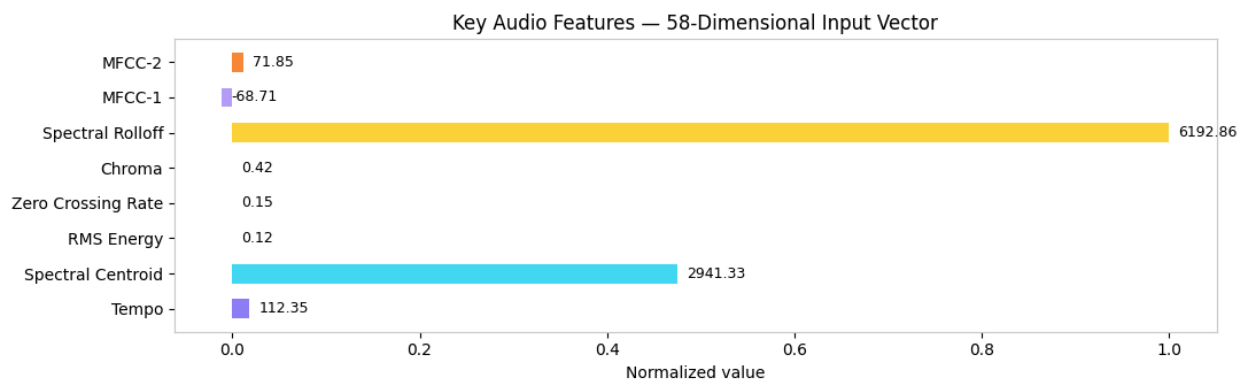
How Neural Networks Work and How I Built Mine

A neural network is a program or model that learns from examples. Instead of being programmed with rigid rules, it finds patterns in data, numerically. They consist of *neurons* arranged in layers. Each receives inputs, multiplying them by a *weight* they choose from training, adds them together, and passes the result through an activation function. Training is simply the mathematical process of finding the right weights through a process of comparing the guesses to the real answer so the network predicts accurately.

The dataset I used was the *GTZAN Genre Collection*, a **standard** benchmark dataset available on Kaggle which contains 1,000 audio clips across 10 genres, 100 songs per genre, each 30 seconds long. I extracted 58 audio features per clip using the *librosa* library: tempo, spectral centroid, RMS energy, zero crossing rate, chroma, spectral rolloff, and 20 MFCC coefficients with their means and variances, all things I have zero clue what actually mean. Finally using those features as the input to the network.

The data then passes through three hidden layers of 256, 128, and 64 neurons using the *ReLU activation function*, $f(x) = \max(0, x)$. Finally, the output layer uses the *Softmax* function to convert raw scores into 10 probabilities that sum to 1.0, representing the model's

confidence in each genre.



I learned to build this through YouTube tutorials mainly, watching people make other types of networks, and a lot of trial and error in Google Colab.

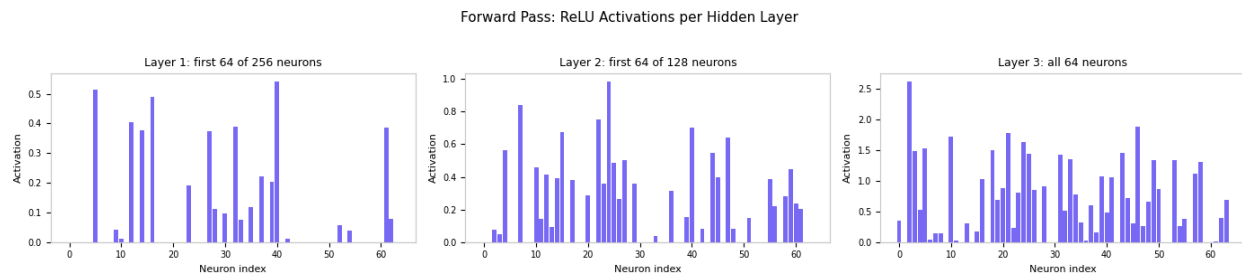
The Calculus I Used

The mathematics behind training a neural network overlaps directly with topics covered in AP Calculus BC. There are four specific connections worth explaining.

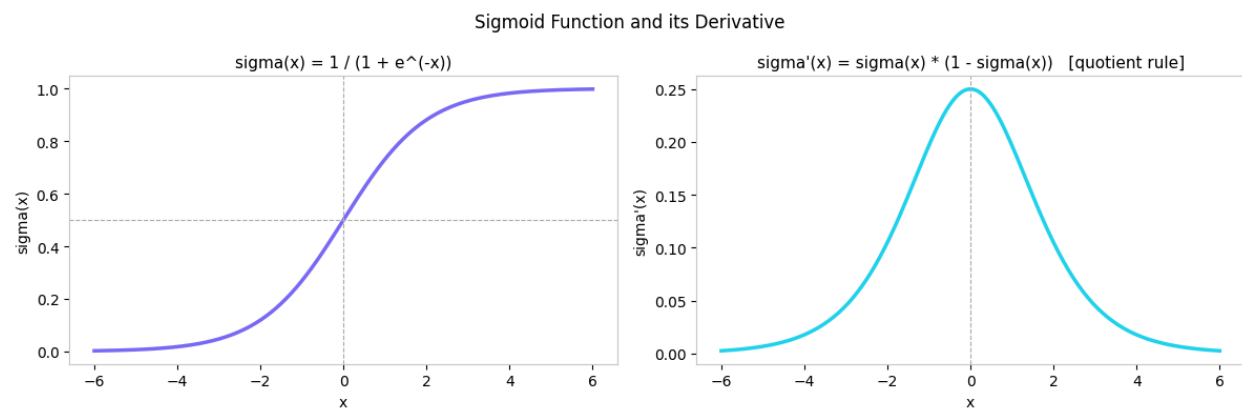
Gradient Descent: The network learns by minimizing a *loss* function, which measures prediction error. Weights are updated using the derivative: $w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial L}{\partial w}$. This derivative points in the direction that increases *loss*, so subtracting it moves the weight toward lower error, just like using $f'(x)$ to find a minimum.

Backpropagation and the Chain Rule: To determine how a weight in an early layer affects the final loss after passing through several layers, the network uses the chain rule:

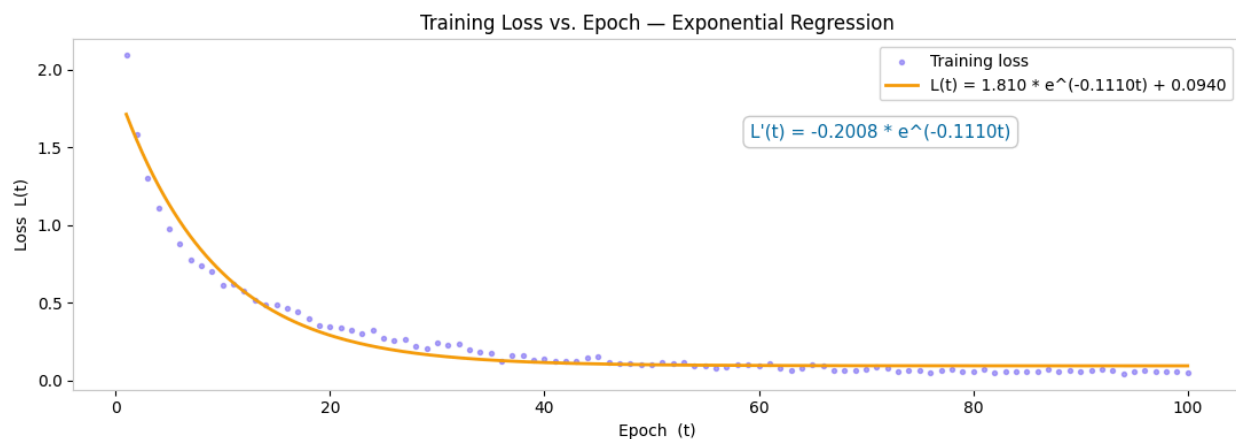
$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a_4} \cdot \frac{\partial a_4}{\partial a_3} \cdot \frac{\partial a_3}{\partial a_2} \cdot \frac{\partial a_2}{\partial w_1}$. Each term is a local derivative. Without the chain rule, deep learning would not work.



Sigmoid Derivative: During backpropagation, the network may need to compute how sensitive a neuron is to its input. For the *sigmoid* function $\sigma(x) = \frac{1}{1 + e^{-x}}$, the derivative is $\sigma'(x) = \sigma(x)(1 - \sigma(x))$. This shows another direct connection between neural networks and differentiation rules.



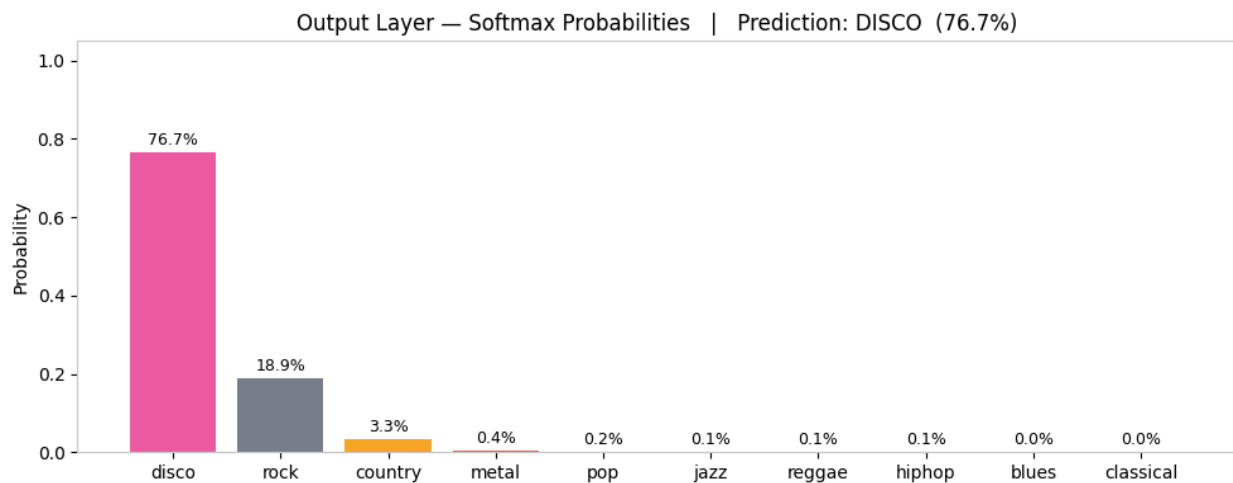
Loss Regression: I modeled my training loss over 100 *epochs* using exponential regression: $L(t) = 1.6870e^{-0.1078t} + 0.0938$. Its derivative, $L'(t) = -0.1818e^{-0.1078t}$, gives the rate of improvement at any epoch. At epoch 10, the model was improving at a rate of -0.0619 loss per epoch. As $t \rightarrow \infty$, loss approaches 0.0938, meaning the model cannot improve much beyond that floor with the same setup.



Results and Personal Assessment

After 100 epochs, basically meaning a training round, my model reached a training accuracy of 98.5% and a test accuracy of 75.5%. This gap shows overfitting, meaning the model learned the training data well, but did not generalize perfectly to newer songs. When tested on a YouTube video of Rick Astley’s song “Never Gonna Give You Up,” it extracted the 58 features

and predicted the genre as disco with 76.7% confidence.



The hardest part of this project was not writing the code, but understanding what the code actually meant. Libraries like TensorFlow make it easy to build a model in a few lines, but that can also turn the network into a black box, making it easy to write, not to understand. There were times I wrote code flawlessly not understanding a thing I'm actually making. Working through the calculus, gradient descent, the chain rule in backpropagation, and the derivative of the loss curve forced the math out of the abstract and helped me make something concrete.

A 75.5% accurate model that can be explained mathematically is more valuable than a more accurate one that cannot be explained at all, especially to a class of high school students.

This project was, in hindsight, a great choice. It gave me exactly the foundation I wanted before studying Data Science, and showed me that the calculus from this class is something I can apply beautifully, whilst being one of the engines behind modern machine learning.

Glossary of Terms

Tools and Technologies

Python: The programming language used to write the code for this project. It is highly readable and currently the industry standard for data science and artificial intelligence.

TensorFlow & Keras: TensorFlow is a massive library (a collection of pre-written code) created by Google for building machine learning models. Keras is an interface built on top of TensorFlow that makes it much easier and more intuitive to use, allowing you to build a neural network in just a few lines of code.

Google Colab: A free service from Google that lets you write and run Python code in your web browser. It is basically Google Docs for coding, and it provides free access to powerful computers (GPUs) needed to train machine learning models.

librosa: A Python library specifically designed for analyzing audio and music. I used it to extract the 58 mathematical features (like tempo and pitch) from the raw audio files.

Datasets and Concepts

"Hello World": In programming, the "Hello World" project is the absolute most basic, beginner-level script you write when learning a new language or concept.

MNIST: The "Hello World" dataset of machine learning. It is a massive collection of 70,000 hand-drawn numbers (0-9). Almost every beginner machine learning tutorial starts by building a model to recognize these handwritten digits.

GTZAN: The dataset I used instead of MNIST. It is a famous benchmark dataset in music analysis, containing 1,000 audio tracks evenly divided across 10 genres (rock, disco, classical, etc.).

Neural Network Components

Neuron (or Node): The basic building block of the network. In an artificial neural network, a neuron is simply a mathematical function that receives a set of numbers as input, does some basic math on them, and passes the result forward.

Weight: A number attached to the connection between two neurons. It acts as a multiplier that determines how important one neuron's signal is to the next neuron. Training the network is literally just the process of adjusting these weights until they produce the right answers.

Activation Function: A math equation that a neuron applies to its input before passing it to the next layer. Without this, the entire network would just be a straight line and couldn't learn complex patterns like music

ReLU (Rectified Linear Unit): The activation function I used in the hidden layers. It is very simple: $f(x) = \max(0, x)$. If the input is negative, it outputs zero. If positive, it passes the number through unchanged.

Softmax: The activation function used on the very last layer. It takes the raw output scores for all 10 genres and squeezes them into probabilities (percentages) that add up to exactly 1.0 (or 100%).

Sigmoid: Another activation function, $\sigma(x) = \frac{1}{1 + e^{-x}}$, that squeezes any number into a range between 0 and 1. While I didn't use it directly as a layer activation, I derived its derivative to show how the network handles sensitivity during training.

Training and Calculus

Epoch: One full training cycle. When the network has looked at every single song in the training dataset exactly one time and updated its weights, that is one epoch. I trained my model for 100 epochs.

Loss: A number representing how "wrong" the model's predictions are. If it is completely guessing, the loss is high. As it learns, the loss drops. The entire goal of training is to minimize this number.

Gradient Descent: The algorithm the network uses to minimize the loss. It calculates the slope (derivative) of the loss curve at its current position, and then takes a step "downhill" by adjusting the weights.

Partial Derivative: Because the network has tens of thousands of weights, the loss function has tens of thousands of variables. A partial derivative calculates the slope of the loss with respect to just one specific weight, pretending all the other weights are frozen.

Backpropagation: The algorithm that calculates all those partial derivatives efficiently. Because the network has multiple layers, a weight in the first layer affects the final loss through a chain reaction. Backpropagation uses the calculus Chain Rule to trace that reaction backward from the output to the input, figuring out exactly how much each weight contributed to the error.